Лабораторная работа 2. Свёрточные нейронные сети

В этой лабораторной работе необходимо будет построить, обучить и улучшить свёрточную нейронную сеть (CNN) для задачи многоклассовой классификации изображений.

Выбор датасета

Вы можете выбрать датасет самостоятельно.

Обязательные требования к датасету

Задача: многоклассовая классификация изображений (≥ 5 классов).

Размер изображений: не более **128×128** пикселей. Если исходные изображения больше или имеют разный размер, приведите их к единому размеру на этапе предобработки.

Объём выборки: от **5 000** до **100 000** изображений суммарно (train + val + test). Если датасет больше, допускается **сабсэмплинг** (с объяснением, как именно).

Каналы: 1 канал (grayscale) или 3 канала (RGB).

Примеры подходящих датасетов

Датасет	Классов	Размер	Объём	Источник
CIFAR-10	10	32×32, RGB	60 000	`torchvision.datasets.CIFAR10`
CIFAR-100	100	32×32, RGB	60 000	`torchvision.datasets.CIFAR100`
Fashion-MNIST	10	28×28, grayscale	70 000	`torchvision.datasets.FashionMNIST`
SVHN	10	32×32, RGB	99 289	`torchvision.datasets.SVHN`
STL-10	10	96×96, RGB	13 000	`torchvision.datasets.STL10`
Flowers102	102	variable (resize)	8 189	`torchvision.datasets.Flowers102`

Подготовка данных

Перед началом экспериментов:

1. Загрузите датасет.
2. Разделите данные на обучающую/валидационную/тестовую выборки, если готового разбиения нет. **Рекомендуемая пропорция:** 80/10/10 (или близко к этому). Зафиксируйте `random_seed`.
3. Приведите изображения к единому размеру. Допускается `Resize((H, W))`. Если важно сохранить пропорции, можно использовать `resize` с сохранением соотношения сторон и `padding`. Опишите выбранный подход.
4. Соберите `Dataset` и `DataLoader`. Для **train**: `shuffle=True`. Для **val/test**: `shuffle=False`.

Задание 1. Обязательное для всех (максимальная оценка — 8)

Постройте и обучите свёрточную нейронную сеть, продемонстрировав владение основными компонентами CNN.

Требования к модели

Модель должна содержать **все перечисленные компоненты** (конкретная архитектура — на ваше усмотрение):

1. **Свёрточные слои** (`nn.Conv2d`) — ≥ 1.
2. **Пулинг** (`nn.MaxPool2d` или `nn.AvgPool2d`) — ≥ 1.
3. **Пакетная нормализация** (`nn.BatchNorm2d`) — ≥ 1.

   Рекомендуемая схема блока: `Conv → BatchNorm → Activation → (Pooling)`.
4. **Функция активации** (`ReLU`, `LeakyReLU`, `ELU` или другая) — после каждого скрытого слоя.

Под «скрытым слоем» здесь понимаются **все слои, кроме выходного** (в том числе `Conv2d` и промежуточные `Linear`).

5. **Полносвязный слой** (`nn.Linear`) — ≥ 1 **помимо выходного**.
6. **Dropout** (`nn.Dropout` или `nn.Dropout2d`) — ≥ 1.

   Допустимо ставить dropout в классификаторе (перед `Linear`) и/или в сверточной части (например, `Dropout2d`). Укажите, где и почему.

   Типичные значения: `p = 0.2–0.5`.
7. **Выходной слой** — полносвязный, число нейронов равно числу классов.
8. **Функция потерь** — кросс-энтропия (`nn.CrossEntropyLoss`).

Требования к обучению

1. Использовать адаптивный оптимизатор. **Рекомендуемый:** Adam.
2. Обучить модель до сходимости.

   Ориентир: 15–30 эпох.

   Если используете early stopping, укажите параметр `patience` (рекомендуется 3–5 эпох без улучшения).
3. Отслеживать на каждой эпохе **train loss** и **val accuracy**
4. Оценить итоговую модель на **тестовом наборе**.
5. Тестовый набор используйте только для финальной оценки модели из Задания 1 (после выбора гиперпараметров по train/val).

Визуализация результатов

Постройте графики изменения **train loss** по эпохам и изменения **val accuracy** по эпохам

Ожидаемый результат

Точность на валидации должна быть **заметно выше случайного угадывания**. Если классов K, то случайное угадывание ≈ 1/K. Как минимальный ориентир: добиться качества хотя бы в **2–3 раза выше** случайного в зависимости от датасета.

Задание 2. Дополнительное (максимальная оценка — 10)

Повысить точность модели, применяя аугментацию данных, более глубокие архитектуры, методы оптимизации и регуляризации.

Ограничения

1. **Запрещено** использовать предобученные модели, пока вы не достигли **80% на валидации** (val accuracy).
2. Тестовый набор используется **только для финальной оценки** и **не используется** для подбора гиперпараметров.

Аугментация данных

Реализуйте аугментацию обучающих данных с помощью `torchvision.transforms`.

Требования:

1. Использовать **не менее двух** различных преобразований аугментации (например, `RandomCrop`, `RandomHorizontalFlip`, `RandomRotation`, `ColorJitter`, `RandomErasing` и т.п.).

   `ToTensor()` и `Normalize()` **не считаются** аугментацией.
2. Выбор преобразований должен быть **обоснован**: объясните, почему эти аугментации подходят для вашего датасета.
3. Для валидационной и тестовой выборки аугментация **не применяется.** Допускаются только детерминированные преобразования (например, resize) и нормализация.
4. Нормализация (`transforms.Normalize`) должна быть применена ко всем выборкам с **одинаковыми** статистиками.

   Статистики (mean/std) берутся из train или из документации датасета.

5. Визуализируйте несколько аугментированных изображений, чтобы проверить корректность преобразований.

Улучшение модели

Проведите **серию экспериментов**, последовательно улучшая модель.

Главное правило: одно изменение за раз.

Возможные направления для экспериментов:

1. Архитектура сети

- Увеличьте количество сверточных слоев (2–4 блока convolution + pooling).
- Постепенно увеличивайте число фильтров: 32 → 64 → 128.
- Экспериментируйте с размерами ядер свёртки (3×3, 5×5).
- Попробуйте различные функции активации: ReLU, LeakyReLU, ELU.
- Добавьте BatchNorm после каждого сверточного слоя.
- Используйте Dropout перед полносвязными слоями.

2. Оптимизация обучения

- Используйте адаптивные оптимизаторы: Adam, RMSprop, AdamW.
- Экспериментируйте с learning rate (0.001, 0.0001).
- Реализуйте early stopping.
- Используйте learning rate scheduler (например, `ReduceLROnPlateau`).
- Подберите размер батча (32, 64, 128).

3. Регуляризация

- Добавьте L2-регуляризацию через параметр `weight_decay` оптимизатора.
- Увеличьте Dropout до 0.5 при появлении переобучения.

Фиксация экспериментов

Для каждого эксперимента заполните строку в таблице. **Val acc** заполняется для каждого эксперимента. **Test acc** — только для baseline и **финальной** модели (или для 1–2 лучших моделей, если вы явно фиксируете правило выбора “лучших” только по val).

№	Что изменено	Архитектура (кратко)	Оптимизатор, lr	Эпох	Val acc	Test acc	Вывод
0	Baseline (Задание 1)	...	Adam, 0.001	20	...%	...%	Стартов точка
1	...	...	...	...	...%	—	...
2	...	...	...	...	...%	—	...
...	...	...	...	...	...	...	...

Ожидаемый результат

Итоговая точность на тесте должна быть **выше**, чем в Задании 1.